

KmerFret & SafeRoute

Marketplace Logistique Intelligente — Cameroun

Rapport de Projet Tuteuré

Conception et développement d'une application mobile de mise en relation logistique avec cartographie participative des routes

Expo React Native TypeScript Spring Boot PostgreSQL

Auteurs :

Ayukegba Evelyne
Ekondo Dawson
Keumeze Christ
Ahmadou Abba'a
Mpah Dada Brayen

Encadrant :

M. Fomekong

Niveau :

Niveau 2 — Tronc Commun

Année académique :

2025–2026

Résumé

Le transport de marchandises au Cameroun souffre de deux dysfonctionnements structurels majeurs : d'une part, l'absence de plateforme centralisée pour la gestion du fret portuaire aux ports de Douala et de Kribi, entraînant des surestaries estimées à plusieurs milliards de Franc CFA (FCFA) par an ; d'autre part, la difficulté des agriculteurs des zones rurales à acheminer leurs récoltes vers les marchés urbains dans des délais garantissant la fraîcheur des produits périssables. Ce rapport présente KmerFret & SafeRoute, une application mobile de mise en relation logistique conçue spécifiquement pour le contexte camerounais. La plateforme connecte chargeurs et transporteurs à travers une marketplace sécurisée intégrant un paiement par séquestre de paiement via Mobile Money, un suivi Global Positioning System (GPS) en temps réel, une cartographie participative des dégradations routières par télémétrie passive et un algorithme de Smart Routing qui adapte les itinéraires à l'état réel des infrastructures. L'architecture repose sur Spring Boot pour le backend, React Native avec Expo pour le frontend mobile et PostgreSQL avec PostGIS pour la persistance géospatiale. Le mode Offline-First garantit le fonctionnement de l'application même dans les zones à faible couverture réseau, caractéristique essentielle sur les axes routiers camerounais.

Mots-clés : application mobile, logistique, mise en relation fret, cartographie participative, Offline-First.

Abstract

Freight transportation in Cameroon suffers from two major structural dysfunctions : on one hand, the absence of a centralised platform for port freight management at the ports of Douala and Kribi, resulting in demurrage fees estimated at several billion CFA Francs per year; on the other hand, the difficulty faced by farmers in rural areas when trying to deliver their harvests to urban markets within freshness-preserving timeframes. This report presents KmerFret & SafeRoute, a mobile logistics matchmaking application specifically designed for the Cameroonian context. The platform connects shippers and carriers through a secure marketplace incorporating Mobile Money escrow payment, real-time GPS tracking, participatory road-degradation mapping through passive telemetry and a Smart Routing algorithm that adapts itineraries to the actual state of road infrastructure. The architecture relies on Spring Boot for the backend, React Native with Expo for the mobile frontend, and PostgreSQL with PostGIS for geospatial persistence. The Offline-First design ensures that the application remains fully operational even in areas with poor network coverage, which is an essential characteristic given the reality of Cameroonian road axes.

Keywords : mobile application, logistics, freight matchmaking, participatory mapping, Offline-First.

Table des figures

2.1	Diagramme de cas d'utilisation de KmerFret & SafeRoute.	15
2.2	Diagramme de classes du domaine métier de KmerFret & SafeRoute. .	15
2.3	Diagramme d'états-transitions du cycle de vie d'une mission.	16
2.4	Diagramme d'activités du flux principal (swimlanes Chargeur / Transporteur).	17
3.1	Écran de connexion de l'application KmerFret (rôle Chargeur).	25
3.2	Tableau de bord du Chargeur avec liste des missions en cours.	26
3.3	Carte participative des dégradations routières sur l'axe Douala–Yaoundé.	27
3.4	Écran de suivi de mission active du Transporteur avec indicateurs télémétrie.	28

Liste des tableaux

1.1	Comparaison des solutions existantes avec KmerFret & SafeRoute . . .	6
2.1	Acteurs du système KmerFret & SafeRoute	8
2.2	Module BF01 — Gestion des Comptes et Authentification	9
2.3	Module BF02 — Création et Gestion des Missions	10
2.4	Module BF03 — Paiement Sécurisé par Séquestre et QR Code	11
2.5	Module BF04 — Télémétrie Passive et Cartographie des Routes	12
2.6	Module BF05 — Smart Routing (Itinéraire Intelligent)	12
2.7	Module BF06 — Mode Offline-First et Synchronisation	13
2.8	Module BF07 — OCR, Alerte de Détresse et Notation	13
2.9	Besoins non fonctionnels — norme ISO/IEC 25010	14
2.10	Couches de l'architecture logique	18
2.11	Mesures de sécurité par niveau	19
2.12	Pile technologique de KmerFret & SafeRoute	20
2.13	Conventions de nommage — KmerFret & SafeRoute	21
2.14	Outils de contrôle qualité	22
3.1	Comparaison des frameworks de développement mobile	23
3.2	Comparaison des frameworks backend	24
3.3	Comparaison des systèmes de gestion de bases de données	24

Abréviations

API Application Programming Interface 3

FCFA Franc CFA i

FCM Firebase Cloud Messaging 8

GPS Global Positioning System i, 4, 5, 7

HTTP HyperText Transfer Protocol 3

JSON JavaScript Object Notation 3

JWT JSON Web Token 3, 9

QR Quick Response 11

REST Representational State Transfer 3

UML Unified Modeling Language 2, 7

Glossaire

marketplace Plateforme numérique biface mettant en relation des offreurs et des demandeurs de services, assurant la transaction et garantissant la confiance entre les parties grâce à des mécanismes de notation et de paiement sécurisé i, 4, 5, 7, I

Offline-First Paradigme de développement dans lequel l'application est conçue pour fonctionner intégralement sans connexion réseau. Les données sont stockées localement (SQLite) puis synchronisées vers le serveur dès le retour de la connectivité i, 2, 7, I

retour à vide Trajet effectué par un camion sans cargaison, généralement sur le trajet de retour après une livraison. Ce phénomène représente une perte économique directe pour le transporteur 1, 4

Smart Routing Algorithme de calcul d'itinéraire intégrant l'état de dégradation des routes (score de dangerosité issu de la télémétrie collective) en plus des critères classiques de distance et de temps de trajet i, 2, 7

surestarie Pénalité financière facturée à un importateur lorsque ses marchandises demeurent dans un port au-delà du délai de franchise accordé, généralement exprimée en FCFA par jour de retard i, 1

séquestre de paiement Mécanisme financier par lequel les fonds d'une transaction sont bloqués sur un compte tiers neutre jusqu'à confirmation de la bonne exécution de la prestation. Dans KmerFret, le séquestre est libéré automatiquement au scan du QR Code de livraison i, 2, 4, 7

télémétrie passive Collecte automatique de données de mouvement (accéléromètre, GPS) en arrière-plan pendant un trajet, sans intervention de l'utilisateur. Elle permet à KmerFret de cartographier les dégradations routières à partir des chocs détectés i, 2, 4, 7, I

Table des matières

Résumé	i
Abstract	ii
Liste des figures	iii
Liste des tableaux	iv
Abréviations	v
Glossaire	vi
Introduction	1
1 Concepts et État de l'Art	3
1.1 Présentation des concepts	3
1.2 État de l'art	5
1.3 Bilan comparatif	6
2 Méthodologie et Conception	7
Introduction du chapitre	7
2.1 Analyse des besoins	8
2.2 Conception UML	14
2.3 Architecture de l'application	18
2.4 Normes de codage et conventions de nommage	21
2.5 Bilan du chapitre	22
3 Implémentation	23
3.1 Choix des technologies	23
3.2 Résultats	25
3.3 Bilan du chapitre	28
Conclusion	I
A Extrait du schéma SQL	II

Introduction

Contexte

L'Afrique subsaharienne connaît depuis le début du XXI^e siècle une croissance démographique et économique soutenue qui exerce une pression grandissante sur ses infrastructures logistiques. Au Cameroun, le secteur du transport de marchandises constitue la colonne vertébrale de l'économie agricole et commerciale du pays. Pourtant, ce secteur reste largement informel : selon un rapport de la Commission Économique pour l'Afrique, plus de 70 % des transactions de transport de fret sur le continent se concluent encore par téléphone ou de bouche-à-oreille, sans traçabilité ni garantie de paiement [cea2022]. Au Port Autonome de Douala, premier port d'Afrique centrale, des milliers de conteneurs transitent chaque semaine et les importateurs cherchent des transporteurs disponibles via leurs réseaux personnels, générant des délais d'attente synonymes de surestaries coûteuses [pad2022]. Dans les zones rurales de l'Ouest camerounais, des tonnes de récoltes périssables (tomates, avocats, café, cacao) pourrissent chaque semaine faute de transport fiable [banquemondiale2023].

Problématique

Face à ces constats, une question centrale émerge : *comment concevoir une solution numérique qui permette de mettre en relation efficacement chargeurs et transporteurs, de sécuriser les transactions financières et de compenser l'insuffisance des infrastructures routières camerounaises, tout en restant fonctionnelle dans des zones à faible connectivité réseau ?*

Motivation

La motivation première de ce projet tient à l'impact social et économique potentiel d'une telle plateforme. Une application de mise en relation logistique pourrait réduire significativement les pertes post-récolte des agriculteurs, diminuer les coûts de transport par l'optimisation des trajets à vide (retour à vides) et générer des données routières utiles aux collectivités territoriales. Par ailleurs, la démocratisation du smartphone et des services Mobile Money au Cameroun crée les conditions favorables à l'adoption massive d'une telle solution [arcep2024].

Objectifs

Ce projet poursuit quatre objectifs principaux :

- O1.** Analyser les besoins fonctionnels et non fonctionnels d'une plateforme de mise en relation logistique adaptée au contexte camerounais.
- O2.** Concevoir une architecture logicielle robuste, sécurisée et Offline-First.
- O3.** Implémenter un prototype fonctionnel couvrant les flux critiques : création de mission, matching, paiement par séquestre de paiement, télémétrie passive passive et Smart Routing.
- O4.** Évaluer les choix technologiques retenus au regard des contraintes spécifiques de l'environnement camerounais.

Plan du rapport

La suite de ce rapport s'organise en trois chapitres. Le Chapitre 1 pose les concepts fondamentaux et dresse un état de l'art des solutions existantes. Le Chapitre 2 détaille la méthodologie d'analyse, la conception Unified Modeling Language (UML) et l'architecture retenue. Le Chapitre 3 présente les choix technologiques justifiés et les résultats obtenus.

Chapitre 1

Concepts et État de l'Art

Ce chapitre introduit les concepts fondamentaux sur lesquels repose KmerFret & SafeRoute, analyse les solutions existantes et les compare dans un tableau synthétique.

1.1 Présentation des concepts

1.1.1 Application mobile (concept technique)

Une application mobile est un logiciel conçu pour s'exécuter sur des appareils portables (smartphones, tablettes) dotés d'un système d'exploitation tel qu'Android ou iOS. Contrairement aux applications de bureau, elle doit composer avec des ressources matérielles limitées (processeur, mémoire vive, batterie), une connectivité réseau intermittente et des modalités d'interaction tactiles [meier2018].

On distingue trois catégories : les applications natives (compilées pour une plateforme spécifique), les applications multi-plateformes (un code unique déployé sur plusieurs systèmes) et les Progressive Web Apps. L'approche multi-plateforme via React Native est retenue dans ce projet pour ses capacités de déploiement simultané Android/iOS depuis une seule base de code TypeScript.

1.1.2 API REST (concept technique)

Une Application Programming Interface (API) de type Representational State Transfer (REST) est une interface de communication entre systèmes informatiques fondée sur le protocole HyperText Transfer Protocol (HTTP) et les principes architecturaux définis par Roy Fielding [fielding2000]. Elle s'appuie sur les verbes HTTP (GET, POST, PUT, DELETE) et représente les ressources sous forme de documents JavaScript Object Notation (JSON).

Les six contraintes REST sont : client-serveur, *statelessness*, mise en cache, interface uniforme, système en couches et code-à-la-demande. Dans KmerFret, l'ensemble des échanges entre l'application mobile et le serveur transite par une API REST sécurisée par des tokens JSON Web Token (JWT), versionnée sous `/api/v1`.

1.1.3 Marché logistique numérique — Marketplace (concept métier)

Une marketplace logistique est une plateforme numérique biface qui met en présence des chargeurs (détenteurs d’une demande de transport) et des transporteurs (offreurs de capacité) [evans2016]. La valeur créée réside dans la réduction des coûts de transaction (recherche, négociation, paiement) et dans la confiance établie grâce aux mécanismes de notation et de séquestre de paiement.

L’économie des effets de réseau s’applique pleinement : plus la plateforme attire de transporteurs, plus elle est attractive pour les chargeurs, et réciproquement. C’est le mécanisme qui a conduit Kobo360 au Nigéria à une croissance rapide après l’atteinte de la masse critique [kobo2023].

1.1.4 Covoiturage de fret et retours à vide (concept métier)

Le covoiturage de fret (*freight pooling*) désigne la mutualisation d’un espace de chargement entre plusieurs expéditeurs ayant des origines et destinations compatibles [dbschenker2022]. Ce concept adresse directement le problème des retour à vides : en Afrique subsaharienne, la situation est structurellement grave faute de systèmes de coordination entre acteurs.

Dans KmerFret, la logique de mise en relation et le filtrage géographique des missions permettent à un transporteur qui vient de livrer à Bafoussam de trouver immédiatement une mission de retour vers Douala, limitant les trajets non productifs.

1.1.5 Géolocalisation et cartographie participative (concept transversal)

La géolocalisation est la technologie permettant de déterminer la position géographique d’un appareil grâce au système GPS [kaplan2005]. La cartographie participative (*crowdsourced mapping*) étend ce concept en agrégeant les données de géolocalisation de multiples utilisateurs pour construire collectivement une représentation de l’environnement.

Ce concept est transversal aux deux problématiques métiers du projet : le suivi de mission et la cartographie des dégradations routières. La télémétrie passive de KmerFret, qui capture les chocs de l’accéléromètre couplés aux coordonnées GPS, constitue une forme de cartographie participative des infrastructures sans effort conscient des chauffeurs.

1.2 État de l'art

1.2.1 Solutions existantes au Cameroun

Cameroon Cargo Exchange (CCE). Plateforme web lancée en 2021, CCE propose un annuaire en ligne permettant aux chargeurs de publier leurs besoins et aux transporteurs de se déclarer disponibles. La mise en relation reste manuelle, aucun paiement n'est intégré et la vérification des documents des transporteurs n'est pas automatisée. L'absence de version mobile native et le non-fonctionnement hors connexion limitent fortement son adoption.

Groupes WhatsApp de fret. Des groupes WhatsApp informels fédèrent des centaines de chauffeurs et de chargeurs sur des axes comme Douala–Yaoundé ou Douala–Bafoussam. Cette solution est gratuite et omniprésente, mais elle ne garantit aucune sécurité de paiement, ne propose aucun suivi de mission et génère une information non structurée, difficilement exploitable.

1.2.2 Solutions existantes à l'international

Uber Freight (États-Unis, Europe). Lancée en 2017, Uber Freight est la référence mondiale de la marketplace de fret numérique. Elle offre une mise en relation instantanée, un paiement garanti et une traçabilité complète. Ses limites dans le contexte africain sont structurelles : l'application n'est pas disponible en Afrique subsaharienne, ne supporte pas les paiements Mobile Money et ne propose pas de mode hors connexion [uberfreight2023].

Kobo360 (Nigéria, Ghana, Kenya, Rwanda). Kobo360 est la solution la plus proche de notre périmètre géographique. Elle connecte chargeurs et transporteurs en Afrique de l'Ouest et de l'Est, intègre un paiement numérique et propose un suivi GPS. Elle ne dispose cependant pas de fonctionnalité de cartographie des routes ni de mode hors connexion, n'est pas déployée au Cameroun et ne prend pas en charge Mobile Money camerounais [kobo2023].

Lori Systems (Kenya, Nigeria, Éthiopie). Lori Systems cible le marché B2B de grande entreprise. Son modèle tarifaire à abonnement mensuel élevé le rend inaccessible aux TPE et agriculteurs. Aucun support du mode hors connexion n'est documenté [lori2023].

Shipa Freight (Moyen-Orient, Afrique du Nord). Shipa Freight propose une interface intuitive pour la réservation de transport mais est absente d'Afrique subsaha-

rienne francophone et ne supporte pas la réalité des routes non asphaltées [shipa2023].

1.3 Bilan comparatif

Le tableau 1.1 compare les solutions identifiées selon dix critères pertinents pour le contexte camerounais.

Table 1.1. Comparaison des solutions existantes avec KmerFret & SafeRoute

Critère	CCE	WA Groups	Uber Freight	Kobo360	Lori Sys- tems	KmerFret
Application mobile native	Non	Non	Oui	Oui	Non	Oui
Mode hors connexion	Non	Non	Non	Non	Non	Oui
Paiement Mobile Money (CM)	Non	Non	Non	Non	Non	Oui
Séquestre de paiement	Non	Non	Oui	Oui	Non	Oui
Suivi GPS en temps réel	Non	Non	Oui	Oui	Oui	Oui
Carte routes dégradées	Non	Non	Non	Non	Non	Oui
Smart Routing (état routes)	Non	Non	Non	Non	Non	Oui
Disponible au Cameroun	Oui	Oui	Non	Non	Non	Oui
Ciblage petits chargeurs	Oui	Oui	Non	Non	Non	Oui
Numérisation OCR documents	Non	Non	Non	Non	Non	Oui

Au vu de ce tableau, aucune application ne remplit tous nos critères. Les solutions internationales sont techniquement avancées mais absentes du marché camerounais et inadaptées aux contraintes locales. Les solutions locales sont accessibles mais rudimentaires, dépourvues de mécanismes de confiance et de paiement sécurisé. KmerFret & SafeRoute se positionne donc sur un créneau vacant, combinant disponibilité locale et fonctionnalités avancées.

Chapitre 2

Méthodologie et Conception

Introduction du chapitre

Ce chapitre présente la démarche méthodologique retenue pour la conception et le développement de KmerFret & SafeRoute. Il définit de manière rigoureuse les exigences fonctionnelles et non fonctionnelles, les modèles de conception UML, l'architecture logicielle ainsi que les conventions de développement adoptées.

À propos de KmerFret & SafeRoute

KmerFret & SafeRoute est une application mobile multiplateforme développée avec Expo et React Native. Elle connecte chargeurs et transporteurs à travers une marketplace sécurisée intégrant un paiement par séquestre de paiement via Mobile Money, un suivi GPS en temps réel, une cartographie participative des dégradations routières par télémétrie passive passive et un algorithme de Smart Routing. Le mode Offline-First garantit le fonctionnement même en zone sans réseau.

2.1 Analyse des besoins

2.1.1 Identification des acteurs

Table 2.1. Acteurs du système KmerFret & SafeRoute

Acteur	Type	Description
Chargeur	Principal	Importateur, agriculteur ou entreprise qui crée une mission, suit sa cargaison, paie via Mobile Money et confirme la livraison par QR Code.
Transporteur	Principal	Propriétaire de camion ou chauffeur qui consulte les missions disponibles, accepte une mission et reçoit le paiement après livraison confirmée.
Administrateur	Principal	Gestionnaire de la plateforme. Supervise les missions, modère les litiges et consulte les tableaux de bord.
Opérateur Mobile	Système externe	MTN MoMo et Orange Money. Leurs API sont appelées pour initier et confirmer les transactions financières.
Système GPS	Système externe	Service de géolocalisation Android pour le suivi en temps réel et le Smart Routing.
Système de notifications	Système externe	Firebase Cloud Messaging (FCM) pour les notifications push.

2.1.2 Besoins fonctionnels

Les besoins fonctionnels sont organisés en sept modules thématiques.

Module BF01 — Gestion des Comptes et Authentification**Table 2.2.** Module BF01 — Gestion des Comptes et Authentification

Réf.	Intitulé	Description
BF01.1	Inscription	L'utilisateur crée un compte avec nom, téléphone (+237XXXXXXXXXX), email optionnel, mot de passe et rôle (Chargeur ou Transporteur).
BF01.2	Vérification OTP	Un code à 6 chiffres est envoyé par SMS. L'utilisateur dispose de 10 minutes pour le saisir.
BF01.3	Connexion JWT	Connexion par numéro + mot de passe. Token JWT (24 h) et refresh token (7 jours) stockés dans l'espace sécurisé de l'appareil.
BF01.4	Profil utilisateur	Modification de la photo de profil, du nom et de la zone géographique.
BF01.5	Profil transporteur	Immatriculation, type de véhicule, capacité en tonnes et documents d'assurance obligatoires.
BF01.6	Réinitialisation MDP	Réinitialisation via OTP SMS. MDP : 8 caractères minimum, 1 majuscule, 1 chiffre.
BF01.7	Déconnexion	Session invalidée côté serveur, token supprimé de l'appareil.

Module BF02 — Création et Gestion des Missions

Table 2.3. Module BF02 — Création et Gestion des Missions

Réf.	Intitulé	Description
BF02.1	Création de mission	Saisie du type de cargaison (PÉRISSABLE / FRET SEC), description, poids, origine, destination, date souhaitée et instructions spéciales.
BF02.2	Estimation tarifaire	Fourchette tarifaire calculée automatiquement selon distance, poids et type de cargaison.
BF02.3	Matching géolocalisé	Notification des Transporteurs compatibles dans un rayon paramétrable autour du point de départ (défaut : 50 km).
BF02.4	Acceptation et contrat	Contrat numérique généré à l'acceptation. Statut : ASSIGNED.
BF02.5	Suivi de statut	Statuts : OPEN, ASSIGNED, IN_TRANSIT, DELIVERED, CANCELLED, DISPUTED.
BF02.6	Historique	Historique complet des missions avec filtres par statut et date.
BF02.7	Annulation	Annulation avant acceptation : remboursement intégral. Après acceptation : médiation déclenchée.

Module BF03 — Paiement Sécurisé par Séquestre et QR Code

Table 2.4. Module BF03 — Paiement Sécurisé par Séquestre et QR Code

Réf.	Intitulé	Description
BF03.1	Initiation du paiement	Choix de l'opérateur (MTN MoMo / Orange Money) et confirmation du montant.
BF03.2	Séquestre automatique	Fonds bloqués après confirmation (<code>payment_status = ESCROWED</code>). Inaccessibles avant livraison confirmée.
BF03.3	Génération QR Code	Token unique (UUID v4) encodé en QR Code affiché par le Transporteur.
BF03.4	Scan et confirmation	Scan du QR Code par le destinataire. Statut : DELIVERED.
BF03.5	Déblocage du paiement	Commission prélevée (7,5 %) et solde net viré au Transporteur en moins de 60 secondes.
BF03.6	Gestion des litiges	QR non scanné sous 48 h : médiation déclenchée automatiquement.
BF03.7	Reçu électronique	Reçu numérique généré et consultable dans l'historique.

Module BF04 — Télémétrie Passive et Cartographie

Table 2.5. Module BF04 — Télémétrie Passive et Cartographie des Routes

Réf.	Intitulé	Description
BF04.1	Activation automatique	Accéléromètre (10 Hz) et GPS (toutes les 5 s) activés en arrière-plan au démarrage d'une mission, sans action du Transporteur.
BF04.2	Détection de chocs	Magnitude $\geq 1,5g$: enregistrement automatique des coordonnées, intensité et horodatage. Délai anti-spam : 2 secondes.
BF04.3	Niveaux de sévérité	LOW ($\geq 1,5g$), MEDIUM ($\geq 2,5g$), HIGH ($\geq 3,5g$), CRITICAL ($\geq 5,0g$).
BF04.4	Stockage local	Enregistrement SQLite si hors ligne, sync PostGIS au retour réseau.
BF04.5	Validation collective	Score de dangerosité croissant si plusieurs chauffeurs signalent le même point.
BF04.6	Carte participative	Carte publique des zones dégradées, filtrable par sévérité et date.

Module BF05 — Smart Routing

Table 2.6. Module BF05 — Smart Routing (Itinéraire Intelligent)

Réf.	Intitulé	Description
BF05.1	Calcul d'itinéraire	Combinaison distance, temps et score de dégradation des routes pour un itinéraire optimal.
BF05.2	Pondération par cargaison	PÉRISSABLE : évitement prioritaire des zones HIGH et CRITICAL . FRET SEC : équilibre distance / dégradation.
BF05.3	Recalcul temps réel	Recalcul instantané en cas d'obstacle signalé.
BF05.4	Affichage comparatif	Deux options proposées : plus court vs recommandé, avec distance, durée et score de dangerosité.

Module BF06 — Mode Offline-First et Synchronisation**Table 2.7.** Module BF06 — Mode Offline-First et Synchronisation

Réf.	Intitulé	Description
BF06.1	Fonctionnement hors connexion	Consultation des missions en cache, enregistrement des incidents, affichage du dernier itinéraire.
BF06.2	File de synchronisation	File SQLite (<code>sync_queue</code>) avec statuts <code>PENDING</code> / <code>DONE</code> / <code>FAILED</code> .
BF06.3	Synchronisation automatique	Sync transparente en arrière-plan dès retour de connectivité.
BF06.4	Envoi en lot	Batch unique pour minimiser la consommation de données.
BF06.5	Indicateur de statut	Indicateur discret du mode hors ligne et items en attente.

Module BF07 — OCR, Détresse et Notation**Table 2.8.** Module BF07 — OCR, Alerte de Détresse et Notation

Réf.	Intitulé	Description
BF07.1	Numérisation OCR	Photo du document physique → extraction de texte → pré-remplissage automatique du formulaire de mission.
BF07.2	Stockage documents	Documents liés à la mission et consultables par les deux parties.
BF07.3	Bouton de détresse	Pression 3 s → SMS chiffré avec position GPS et ID mission, même sans internet.
BF07.4	Notation post-mission	Notation mutuelle 1–5 étoiles alimentant le score de confiance des profils.

2.1.3 Besoins non fonctionnels

Table 2.9. Besoins non fonctionnels — norme ISO/IEC 25010

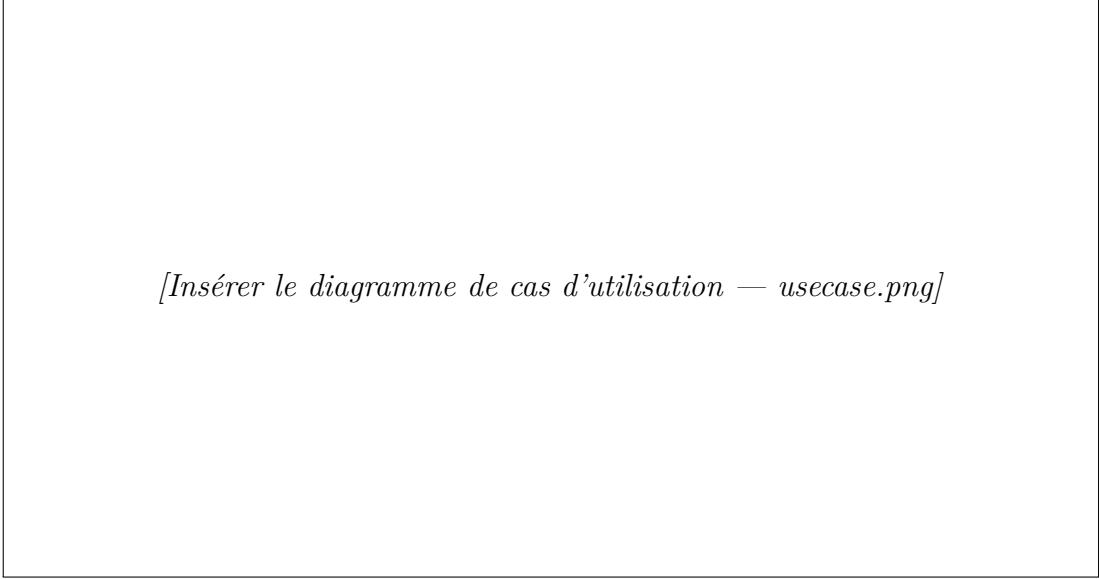
Réf.	Critère (ISO 25010)	Exigence
BNF01	Performance	Écran principal affiché en moins de 3 s sur 3G. Requêtes API répondent en moins de 2 s.
BNF02	Disponibilité	Back-end : 99 % de disponibilité. Fonctionnement hors connexion intégral pour les fonctions critiques.
BNF03	Sécurité	Communications chiffrées TLS 1.3. Mots de passe hachés BCrypt (facteur 12). JWT valable 24 h.
BNF04	Confidentialité	Géolocalisation strictement opt-in. Aucune position conservée après clôture de la mission.
BNF05	Utilisabilité	Interface Material Design 3. Inscription en 4 écrans maximum. Interface en français.
BNF06	Maintenabilité	Couverture tests unitaires : 60 % minimum. Code documenté JSDoc / Javadoc. Principes SOLID respectés.
BNF07	Portabilité	Base de code unique compilée Android via EAS Build (.aab). iOS prévu en phase 2.
BNF08	Évolutivité	Nouveau fournisseur de paiement intégrable sans re-fonte. API versionnée /api/v1.
BNF09	Compatibilité matérielle	Fonctionne sur Android avec 2 Go de RAM minimum.
BNF10	Consommation réseau	Synchronisation en lots. Application installée : 80 Mo maximum.

2.2 Conception UML

2.2.1 Diagramme de cas d'utilisation

Le diagramme de la figure 2.1 modélise les interactions entre les trois acteurs principaux et les fonctionnalités de l'application. Le Chargeur interagit principalement avec les fonctions de création de mission et de paiement ; le Transporteur avec les

fonctions de consultation, d'acceptation et de suivi ; l'Administrateur avec les fonctions de supervision et de modération.

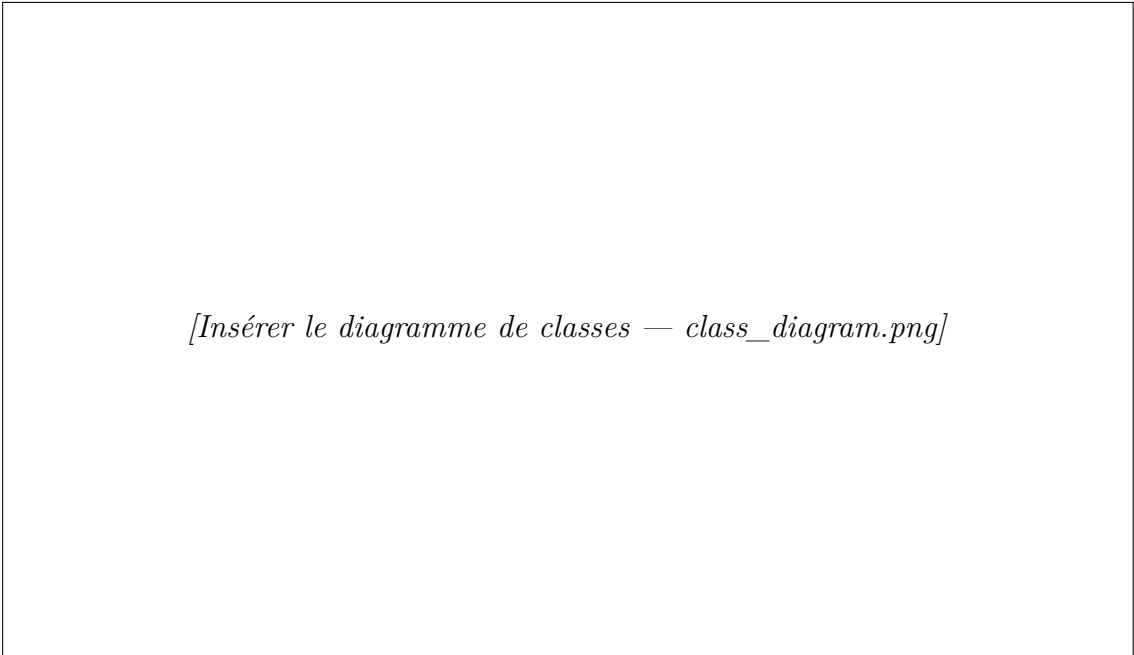


[Insérer le diagramme de cas d'utilisation — usecase.png]

Figure 2.1. Diagramme de cas d'utilisation de KmerFret & SafeRoute.

2.2.2 Diagramme de classes

Le diagramme de la figure 2.2 représente les entités du domaine métier. Les entités centrales sont `User` (polymorphe par `role`), `Mission`, `Truck`, `RoadHazard` et `TelemetryPosition`. Une mission est liée à un Chargeur (`importer_id`) et optionnellement à un Transporteur (`driver_id`) et un camion (`truck_id`).



[Insérer le diagramme de classes — class_diagram.png]

Figure 2.2. Diagramme de classes du domaine métier de KmerFret & SafeRoute.

2.2.3 Diagramme d'états-transitions

La figure 2.3 modélise le cycle de vie d'une mission. Les états sont : OPEN (publiée), ASSIGNED (transporteur accepté, séquestre actif), IN_TRANSIT (en cours, télémétrie active), DELIVERED (QR scanné, paiement débloqué), CANCELLED et DISPUTED.

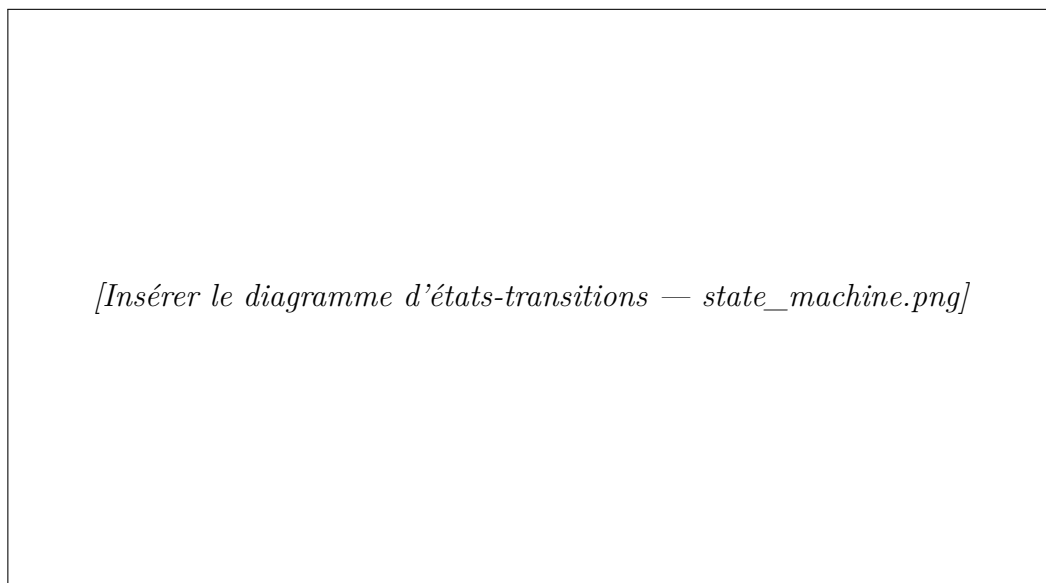


Figure 2.3. Diagramme d'états-transitions du cycle de vie d'une mission.

2.2.4 Diagramme d'activités

La figure 2.4 illustre le flux d'activités du scénario principal en swimlanes (Chargeur / Transporteur). Le point de synchronisation central se produit à la confirmation du paiement en séquestre, après quoi le transport peut démarrer. Le deuxième point de synchronisation est le scan du QR Code.

[Insérer le diagramme d'activités — activity_diagram.png]

Figure 2.4. Diagramme d'activités du flux principal (swimlanes Chargeur / Transporteur).

2.3 Architecture de l'application

2.3.1 Architecture logique

Table 2.10. Couches de l'architecture logique

Couche	Responsabilité
Présentation	Écrans React Native, composants Expo, navigation React Navigation. Aucune logique métier.
Logique métier (mobile)	Hooks personnalisés, services télémétrie / synchronisation / détresse. Orchestre les appels API et met à jour l'état global (Zustand).
Accès données (client)	Couche Axios + intercepteurs JWT. Gestion des tokens et basculement hors ligne.
Persistance locale	Expo SQLite : missions en cache, positions GPS, incidents routiers, file de synchronisation.
Logique métier (serveur)	Spring Boot 3.2 : validation des entrées, orchestration des services (paiement, matching, smart routing).
Persistance serveur	PostgreSQL 15 + PostGIS 3.4 : données relationnelles et géospatiales.

2.3.2 Architecture physique

L'application mobile est distribuée via le Google Play Store au format `.aab` (EAS Build). Le serveur Spring Boot et la base de données PostgreSQL sont conteneurisés via Docker et déployables sur un cloud (AWS, Azure ou VPS africain). Les communications entre le mobile et le serveur transitent exclusivement par HTTPS sur le port 443. MTN MoMo, Orange Money et Firebase sont des systèmes externes appelés depuis des adaptateurs dédiés côté serveur.

2.3.3 Architecture sécuritaire

Table 2.11. Mesures de sécurité par niveau

Niveau	Mesure appliquée
Transport réseau	Chiffrement TLS 1.3 sur toutes les communications réseau.
Authentification	JWT signé HMAC-SHA256 (24 h) + refresh token (7 jours) stocké dans <code>expo-secure-store</code> .
Autorisation	Contrôle d'accès par rôles (<code>CHARGEUR</code> , <code>TRANSPORTEUR</code> , <code>ADMIN</code>) via Spring Security.
Données au repos	Mots de passe hachés BCrypt (facteur 12). QR tokens UUID v4 non prédictibles.
Paieement	Fonds séquestrés inaccessibles sans validation du token QR côté serveur.
Alerte détresse	SMS signé HMAC-SHA256 (clé partagée), authentifié sans internet.

2.3.4 Architecture technologique

Table 2.12. Pile technologique de KmerFret & SafeRoute

Couche	Technologie	Rôle
Backend	Spring Boot 3.2 (Java 17)	API REST, sécurité JWT, logique métier
ORM	Hibernate / Spring Data JPA	Persistance objet-relationnel
Base de données	PostgreSQL 15 + PostGIS 3.4	Données métier et géospatiales
Frontend mobile	React Native + Expo SDK 51	Application Android/iOS
UI mobile	React Native Paper (MD3)	Composants Material Design 3
BD locale	Expo SQLite	Persistance hors connexion
Gestion d'état	Zustand + TanStack Query	État global et cache serveur
Authentification	JWT (JJWT 0.12) + Spring Security	Sécurisation et rôles
Paieement	MTN MoMo API / Orange Money API	Mobile Money camerounais
Notifications	Firebase Cloud Messaging	Notifications push
Conteneurisation	Docker + Docker Compose	Orchestration serveur
Build mobile	EAS Build → .aab	Publication Google Play

2.4 Normes de codage et conventions de nommage

2.4.1 Conventions de nommage

Table 2.13. Conventions de nommage — KmerFret & SafeRoute

Élément	Convention	Exemple
Composant React Native	PascalCase	CarteMission, BoutonDetresse
Variable / paramètre	camelCase	missionActive, magnitudeChoc
Constante globale	SCREAMING_SNAKE_CASE	SENSE_CHOC_LOW, INTERVALLE_GPS_MS
Fonction utilitaire	camelCase, verbe + nom	calculerDistance(), formaterFcfa()
Hook React	use + PascalCase	useMission(), useTelemetrie()
Interface TypeScript	PascalCase	Mission, IncidentRoutier
Énumération TypeScript	PascalCase (membres SCREAMING)	enum StatutMission { OPEN, IN_TRANSIT }
Fichier composant	PascalCase + .tsx	CarteMission.tsx
Fichier service	camelCase + .ts	telemetryService.ts
Classe Java	PascalCase	MissionService, HazardController
Colonne SQL	snake_case	driver_id, shock_magnitude
Branche Git	kebab-case conventionnel	feature/smart-routing
Endpoint API	kebab-case pluriel	/api/v1/road-hazards

2.4.2 Outillage qualité

Table 2.14. Outils de contrôle qualité

Outil	Rôle et configuration
ESLint	Analyse statique TypeScript. Aucune erreur tolérée au commit.
Prettier	Formatage automatique (tabWidth : 2, guillemets simples).
TypeScript strict	Mode <code>strict</code> activé. Aucun <code>any</code> implicite.
Jest + Testing Library	Tests unitaires. Couverture cible : 60 %.
Husky + lint-staged	Pre-commit : ESLint + Prettier + tests. Commit rejeté si échec.
Commitizen	Conventional Commits : <code>feat(telemetry): ajouter détection CRITICAL</code>
Swagger / OpenAPI 3.0	Documentation automatique de l'API sur <code>/api/docs</code> .

2.5 Bilan du chapitre

Synthèse des livrables du Chapitre 2

- Analyse des besoins fonctionnels** : sept modules (BF01 à BF07) couvrant l'authentification, les missions, le paiement, la télémétrie, le Smart Routing, le mode hors connexion et les fonctions transversales.
- Analyse des besoins non fonctionnels** : dix critères (BNF01 à BNF10) selon ISO/IEC 25010.
- Conception UML** : quatre diagrammes (cas d'utilisation, classes, états-transitions, activités).
- Architecture** : logique en couches, physique (Docker / cloud), sécuritaire (TLS, JWT, BCrypt) et technologique (pile complète).
- Normes et conventions** : 13 catégories de nommage et outillage qualité définis pour toute l'équipe.

Chapitre 3

Implémentation

3.1 Choix des technologies

3.1.1 Framework mobile : React Native vs Flutter vs Ionic

Table 3.1. Comparaison des frameworks de développement mobile

Critère	React Native	Flutter	Ionic
Langage	TypeScript / JS	Dart	TypeScript / JS
Courbe d'apprentissage	Faible	Modérée	Faible
Rendu natif	Oui	Oui (Skia)	Non (WebView)
Performances	Élevées	Très élevées	Moyennes
Écosystème	Très riche	Riche	Moyen
Support Expo SQLite	Oui	Limité	Non
Support Expo Sensors	Oui	Partiel	Non
Communauté	>200 k tags SO	≈80 k	≈25 k

Choix retenu : React Native avec Expo. Flutter présente de meilleures performances brutes mais son écosystème offline (Hive, Drift) est moins mature qu'Expo SQLite pour notre cas d'usage. Ionic utilise un WebView et dégrade les performances sur les appareils d'entrée de gamme répandus au Cameroun. React Native bénéficie d'un écosystème très riche (expo-location, expo-sensors, react-native-paper) et d'une courbe d'apprentissage compatible avec les compétences TypeScript de l'équipe.

3.1.2 Framework backend : Spring Boot vs Django vs Node.js Express

Table 3.2. Comparaison des frameworks backend

Critère	Spring Boot	Django REST	Node.js Express
Langage	Java	Python	JavaScript
Typage statique	Oui	Partiel	Non
Sécurité intégrée	Spring Security	django-allauth	Manuelle
Intégration PostGIS	Hibernate Spatial	GeoDjango	sequelize-pg
Transactions ACID	Oui	Oui	Limitée
Maturité production	Très haute	Haute	Moyenne

Choix retenu : Spring Boot 3.2. Le typage statique de Java réduit les erreurs à l'exécution dans un système de paiement où la fiabilité est critique. Spring Security fournit nativement le contrôle d'accès par rôles sans développement supplémentaire. Hibernate Spatial offre un support natif de PostGIS. Node.js Express manque de maturité pour les systèmes financiers critiques et Django n'offre pas la même robustesse transactionnelle pour la gestion du séquestre.

3.1.3 Base de données : PostgreSQL vs MySQL vs MongoDB

Table 3.3. Comparaison des systèmes de gestion de bases de données

Critère	PostgreSQL	MySQL	MongoDB
Extension géospatiale	PostGIS (Oui)	Limitée	Géo 2D seulement
Transactions ACID	Oui	Oui (InnoDB)	Depuis v4.0
Colonnes calculées	Oui (STORED)	Oui	Non
Requêtes ST_Within	Oui	Non	Non
Licence	BSD (libre)	GPL/Commerciale	SSPL

Choix retenu : PostgreSQL 15 avec PostGIS. PostGIS transforme PostgreSQL en base de données géospatiale complète. Les fonctions `ST_Within` et `ST_Distance` sont

indispensables pour la carte des incidents et le Smart Routing. MySQL ne supporte pas PostGIS et MongoDB ne permet pas les requêtes spatiales avancées nécessaires à notre périmètre fonctionnel. Les colonnes calculées (`GENERATED ALWAYS AS STORED`) permettent de dériver automatiquement la commission sans logique applicative supplémentaire.

3.2 Résultats

Cette section présente les principales interfaces réalisées lors de la phase d'implémentation. Chaque figure est référencée par son numéro.

3.2.1 Module d'authentification

La figure 3.1 illustre l'écran de connexion. L'interface adopte la charte Material Design 3 avec le vert primaire (`#1B5E20`) et l'orange (`#E65100`). Le champ numéro est pré-formaté `+237XXXXXXXX` et le bouton est désactivé tant que la validation côté client n'est pas passée.

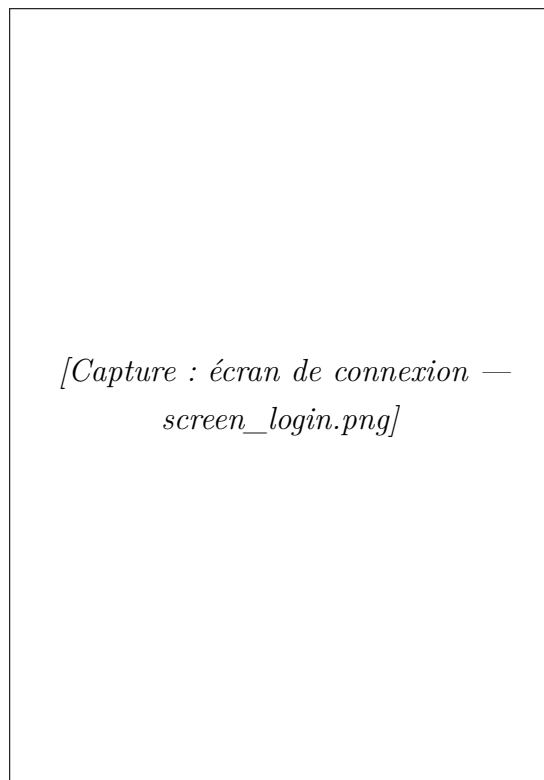


Figure 3.1. Écran de connexion de l'application KmerFret (rôle Chargeur).

3.2.2 Tableau de bord du Chargeur

La figure 3.2 présente le tableau de bord du Chargeur. Il affiche la liste des missions actives sous forme de cartes colorées selon le statut (**OPEN** en bleu, **IN_TRANSIT** en orange, **DELIVERED** en vert), un bouton flottant pour créer une mission et un indicateur de synchronisation en temps réel.

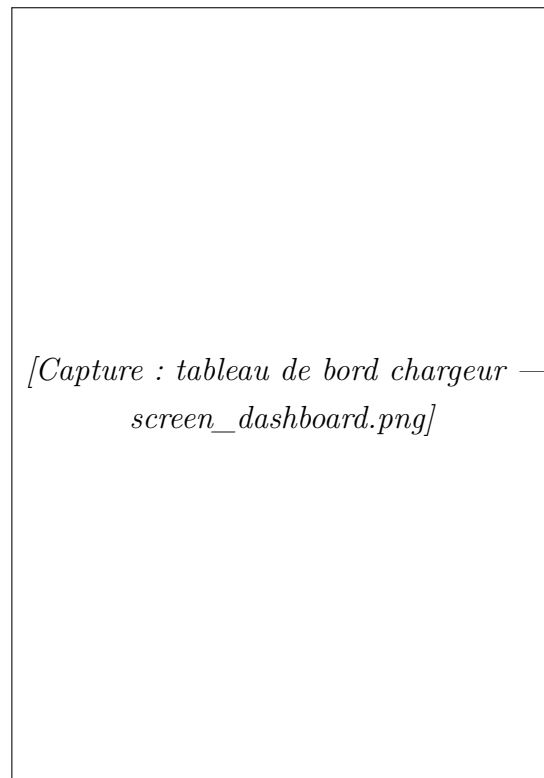


Figure 3.2. Tableau de bord du Chargeur avec liste des missions en cours.

3.2.3 Carte télémétrique des incidents routiers

La figure 3.3 illustre la carte participative. Les points rouges indiquent les zones **HIGH** ou **CRITICAL**, les points orange les zones **MEDIUM**. Chaque marqueur est cliquable et affiche le nombre de signalements confirmés et la date du dernier relevé.

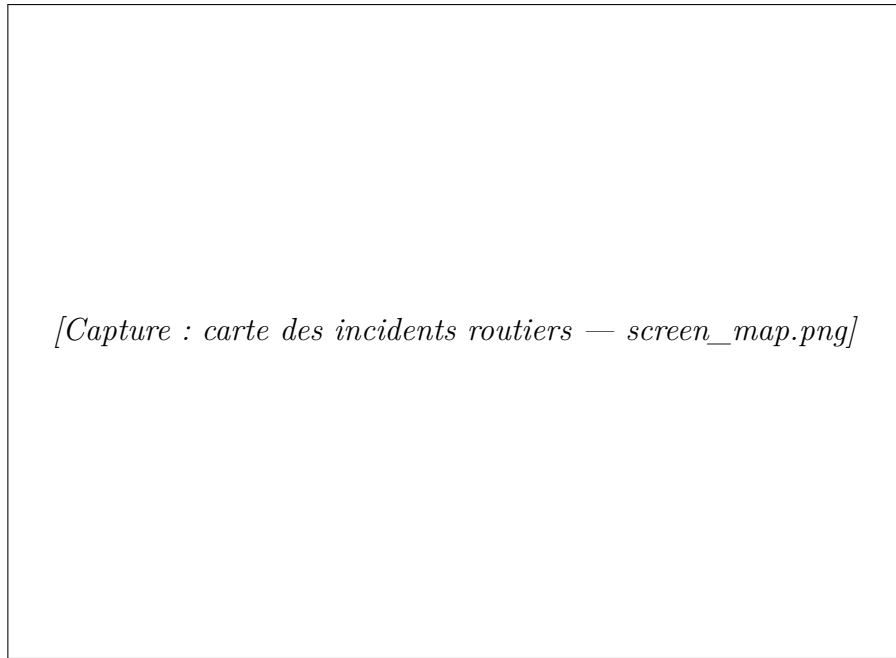


Figure 3.3. Carte participative des dégradations routières sur l’axe Douala–Yaoundé.

3.2.4 Écran de mission active (Transporteur)

La figure 3.4 montre l’interface du Transporteur pendant un trajet. Elle affiche la position GPS en temps réel, la vitesse actuelle, le compteur de chocs depuis le départ, le bouton de détresse (rouge, 3 s de pression requises) et l’indicateur de télémétrie active.

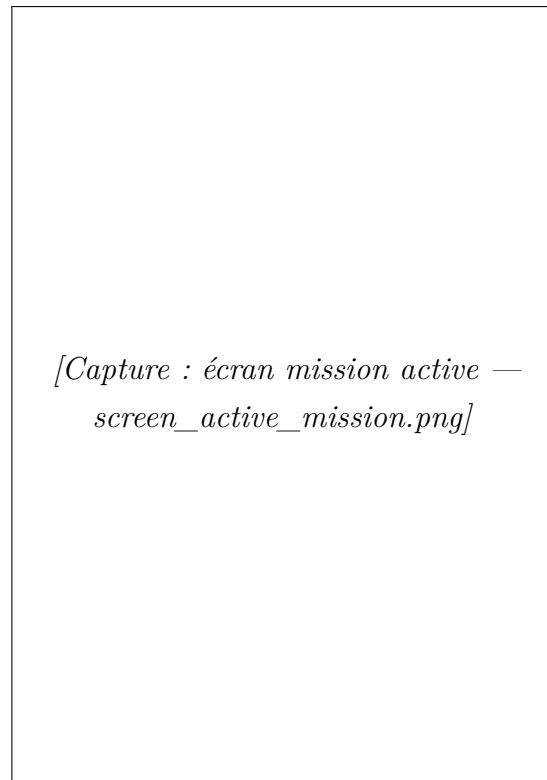


Figure 3.4. Écran de suivi de mission active du Transporteur avec indicateurs télémétrie.

3.3 Bilan du chapitre

Synthèse des livrables du Chapitre 3

- 1. Choix technologiques justifiés** : comparaison objective de trois technologies pour chaque aspect (mobile, backend, base de données). Chaque alternative présente des désavantages objectifs qui justifient le choix retenu.
- 2. Résultats d'implémentation** : quatre captures d'écran commentées couvrant l'authentification, le tableau de bord, la carte télémétrique et l'écran de mission active.

Conclusion

Ce projet a permis de concevoir et d'implémenter un prototype fonctionnel de KmerFret & SafeRoute, une marketplace logistique mobile adaptée aux réalités du Cameroun. L'état de l'art a confirmé l'absence de solution couvrant simultanément la mise en relation chargeur-transporteur, le paiement par séquestre via Mobile Money, la cartographie participative des routes et le mode hors connexion.

L'architecture retenue (Spring Boot, React Native, PostgreSQL/PostGIS) s'est révélée cohérente avec les contraintes identifiées. Le mode Offline-First, garanti par la combinaison SQLite embarqué et service de synchronisation automatique, répond à la réalité des axes routiers à faible couverture réseau. La télémétrie passive constitue l'avantage concurrentiel différenciant : elle génère des données exclusives sur l'état des routes sans contraindre les utilisateurs.

Les perspectives d'évolution portent sur trois axes : l'amélioration de l'algorithme de Smart Routing (intégration d'un moteur de graphe enrichi des scores de dégradation), la monétisation des données routières auprès des collectivités et des assureurs, et l'extension géographique vers les marchés CEMAC voisins (Gabon, Congo, RCA).

Annexe A

Extrait du schéma SQL

```
[language=SQL, caption=Extrait de la table missions (init.sql).] CREATE TABLE
missions ( id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), importer_id UUID NOT NULL REFERENCES
commission_rate) STORED, driver_payout DECIMAL(12, 2) GENERATED ALWAYS AS (total_price -
total_price * commission_rate) STORED, status VARCHAR(20) DEFAULT 'OPEN' CHECK (status IN
```